Aplicaciones con Enfoque Orientado a Servicios

Tema Nº13:
Reglas de negocio

Indicador de logro Nº13:
Implementa reglas de negocio mediante JBoss Rules (Drools)

idat

TEMA Nº13:
Reglas de negocio

Subtema 13.1:
¿Qué es una regla de negocio?

Una regla de negocio es una condición que se debe satisfacer cuando se realiza una actividad de negocio. Una regla puede imponer una política de negocio, tomar una decisión o inferir nuevos datos de datos existentes.

Ejemplos:

Regla de negocio para cotizar precios determinados para los clientes en función del estado del cliente

Un proveedor establecer el estado preferido de cada cliente y crea una regla de negocio para determinar cuánto cuesta a cada cliente el mismo objeto. A continuación, se indican términos básicos que definen los bloques de creación de cada regla de negocio:

- **Condiciones**
  Una condición describe una situación o estado que se debe colocar en orden para que ocurra una acción determinada. En este ejemplo, la condición es el estado del cliente.

- **Acciones**
  Una acción es el suceso resultante de la evaluación de la condición. Cada acción se enlaza únicamente con la condición que le precede. En este ejemplo, la acción es la cotización del precio; es diferente para cada una de las condiciones que se encuentran.

Estado del cliente	Precio de cotización
Bronce	S/ 5
Plata	S/ 4
Oro	S/ 3

La interacción entre las condiciones y las acciones determina la forma de la regla de negocio. Si la condición se cumple, se realiza la acción.

Subtema 13.2:
Drools Workbench

En la mayoría de las empresas la lógica de negocio se encuentra dispersa por diferentes sitios:

- En el código de las aplicaciones
- Hojas de cálculo
- En la mente de los expertos en la materia

Este hecho hace que, a la mayoría de las personas de la organización les sea complejo consultar y comprender las reglas que constituyen la base del negocio.

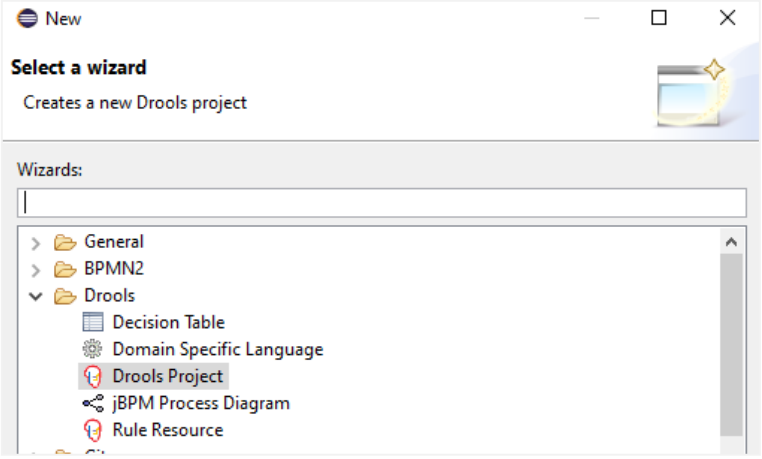
Los sistemas de gestión de reglas de negocio (BRMS) como Drools surgen ante la necesidad de centralizar y gestionar la lógica de negocio. Para ello, se codifica dicha lógica en forma de reglas de negocio, que sirven para tomar decisiones dentro de un contexto.

Las reglas se ejecutan dentro de un motor de reglas (BRM). Las reglas, por tanto, no sirven únicamente para representar la lógica de negocio, sino también para ejecutarla.

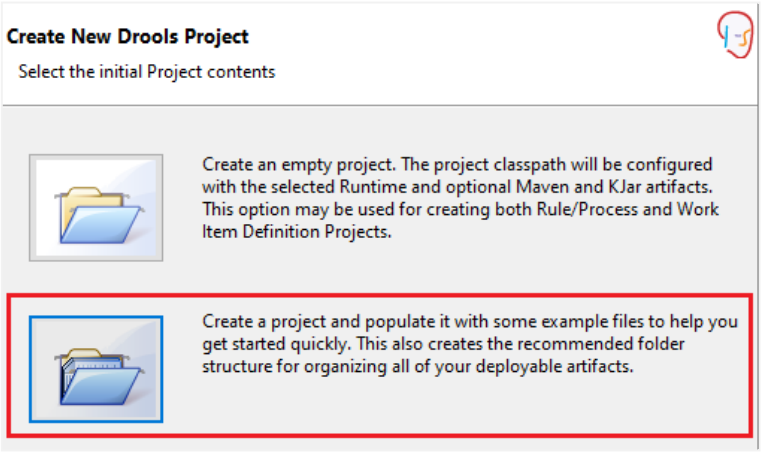
Además, el hecho de que la lógica se encuentre codificada en una regla, hace más fácil su comprensión que cuando se encuentra en el código de una aplicación, sobre todo para el personal no técnico.

Ejemplos:

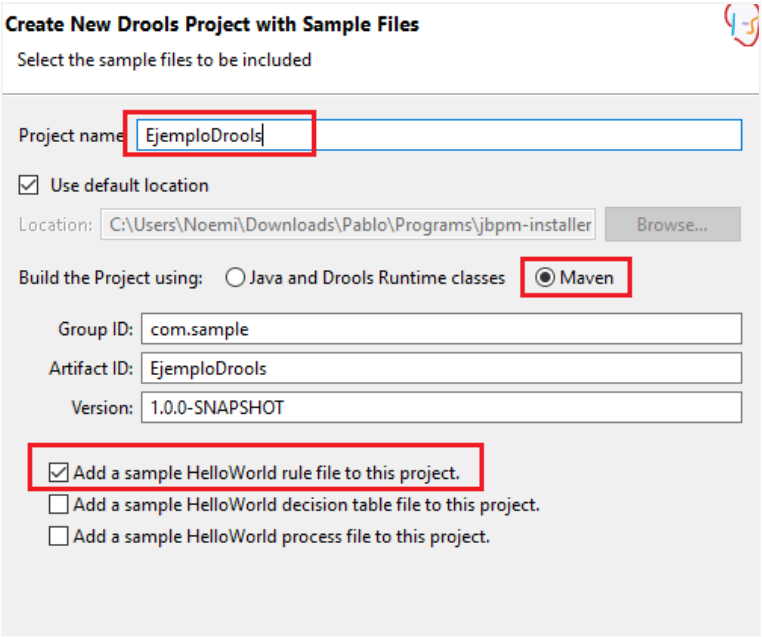
1. Para crear un proyecto desde cero, seleccione la opción *File* > *New Project*, y seleccione la opción *Drools Project* de la sección *Drools*.



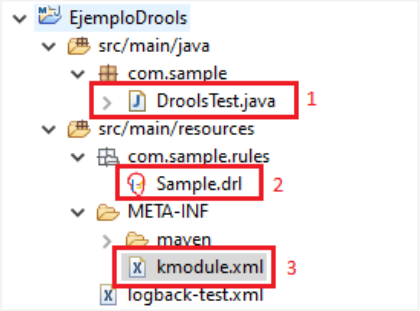
2. Elija la opción de proyecto con ejemplo.



3. Ingrese el nombre del proyecto, manteniendo la configuración de Maven y agregar un ejemplo de regla de negocio.



4. Verifique la estructura del proyecto. El archivo DroolsTest.java contiene el código para ejecutar las reglas de negocio; Sample.drl contiene las reglas de negocio; y kmodule.xml contiene la configuración para lectura del paquete *rules*.



5. El archivo Sample.drl contiene las reglas de negocio.

DroolsTest.java | Sample.drl | kmodule.xml

```
package com.sample.rules

import com.sample.DroolsTest.Message;

rule "Hello World"
    when
        m : Message( status == Message.HELLO, myMessage : message )
    then
        System.out.println( myMessage );
        m.setMessage( "Goodbye cruel world" );
        m.setStatus( Message.GOODBYE );
        update( m );
end

rule "GoodBye"
    when
        Message( status == Message.GOODBYE, myMessage : message )
    then
        System.out.println( myMessage );
end
```

6. Ejecute DroolsTest como proyecto Java y verifique el resultado en la consola.

```
<terminated> DroolsTest (1) [Java Application] C:\Program Files\Java\jre1.8.0_20
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBind
SLF4J: Defaulting to no-operation (NOP) logger implementatio
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinde
Hello World
Goodbye cruel world
```

Subtema 13.3:

Implementación de reglas de negocio con jBPM y JBoss Rules

A un servicio jBPM se le pueden agregar reglas de negocio.

Ejemplos:

El proyecto se continuará con el proyecto jBPM del tema 12.

1. Añada la clase Usuario.java dentro de la carpeta com.sample.

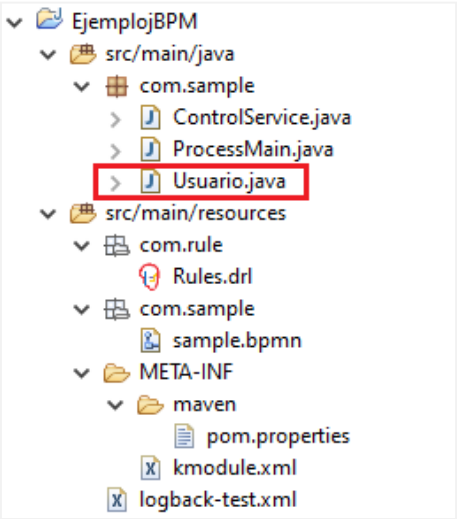
```
package com.sample;

public class Usuario {

    private String usuario;
    private String permisoUrlApi;
    public String getUsuario() {
        return usuario;
    }
    public void setUsuario(String usuario) {
        this.usuario = usuario;
    }
    public String getPermisoUrlApi() {
        return permisoUrlApi;
    }
    public void setPermisoUrlApi(String permisoUrlApi) {
        this.permisoUrlApi = permisoUrlApi;
    }

}
```

2. En la carpeta *src / main / resources* agrega la carpeta com.rule. Añada el archivo **Rules.drl**.



3. Añada la siguiente regla de negocio: solo el usuario *admin* puede ingresar al servicio expuesto por ESB Mule.

```
Rules.drl   Usuario.java   ControlService.java
1  package com.rule
2
3  import com.sample.Usuario
4
5  rule "Url para Usuario admin"
6      when
7          usuarioObject: Usuario(usuario=="admin")
8      then
9          usuarioObject.setPermisoUrlApi("http://localhost:8081/api/productos");
10     end
11 rule "Url para Usuario vendedor"
12     when
13         usuarioObject: Usuario(usuario=="visita")
14     then
15         usuarioObject.setPermisoUrlApi("");
16     end
```

4. Configure el archivo kmodule.xml para ubicar el paquete de reglas *com.rule*.

```
Rules.drl   Usuario.java   ControlService.java   kmodule.xml
1  <kmodule xmlns="http://jboss.org/kie/6.0.0/kmodule">
2
3    <kbase name="kbase" packages="com.sample"/>
4
5    <kbase name="rules" packages="com.rule">
6        <ksession name="ksession-rules"></ksession>
7    </kbase>
8
9  </kmodule>
```

7. Modifique la clase ControlService.java añadiendo el método *obtenerUrl* con la configuración del usuario **visitante**.

```
Rules.drl   Usuario.java   ControlServic...   kmodule.xml   ControlSalud...
36         } catch (Exception e) {
37             System.out.println("Exception in NetClientGet:- " + e);
38         }
39     }
40
41     private static String obtenerUrl(){
42         String url = "";
43         try {
44             KieServices ks = KieServices.Factory.get();
45             KieContainer kContainer = ks.getKieClasspathContainer();
46             KieSession kSession = kContainer.newKieSession("ksession-rules");
47
48             Usuario usuario = new Usuario();
49             usuario.setUsuario("visitante");
50
51             kSession.insert(usuario);
52             kSession.fireAllRules();
53
54             url = usuario.getPermisoUrlApi();
55         } catch (Throwable t) {
56             t.printStackTrace();
57         }
58         return url;
59     }
60
61 }
```

8. Modifique el método apiProductosFromMuleESB de la clase ControlService.java.

```
Rules.drl   Usuario.java   ControlServic...   kmodule.xml   ControlSalud...

13
14      public static void apiProductosFromMuleESB(){
15          System.out.println("Hola api from Mule");
16          String str_url = obtenerUrl();
17          if(str_url == null || str_url.isEmpty()) return;
18          try {
19
20              URL url = new URL(str_url);
21              HttpURLConnection conn = (HttpURLConnection) url.openConnection();
22              conn.setRequestMethod("GET");
23              conn.setRequestProperty("Accept", "application/json");
24              if (conn.getResponseCode() != 200) {
25                  throw new RuntimeException("Failed : HTTP Error code : "
26                          + conn.getResponseCode());
27              }
28              InputStreamReader in = new InputStreamReader(conn.getInputStream());
29              BufferedReader br = new BufferedReader(in);
30              String output;
31              while ((output = br.readLine()) != null) {
32                  System.out.println(output);
33              }
34              conn.disconnect();
35
36          } catch (Exception e) {
37              System.out.println("Exception in NetClientGet:- " + e);
38          }
39      }
```

9. Ejecute el aplicativo y verifique el resultado de la consola.

```
jun 18, 2020 4:48:38 PM org.kie.api.internal.utils.Ser
INFORMACIÓN: Loading kie.conf from  jar:file:/C:/Users
jun 18, 2020 4:48:38 PM org.kie.api.internal.utils.Ser
INFORMACIÓN: Adding Service +org.optaplanner.core.impl

jun 18, 2020 4:48:40 PM org.jbpm.runtime.manager.impl.
INFORMACIÓN: SingletonRuntimeManager is created for de
Hola api from Mule
jun 18, 2020 4:48:40 PM org.drools.compiler.kie.builde
INFORMACIÓN: Found kmodule: file:/C:/Users/Noemi/Downl
jun 18, 2020 4:48:41 PM org.drools.compiler.kie.builde
INFORMACIÓN: KieModule was added: FileKieModule[releas
jun 18, 2020 4:48:41 PM org.drools.compiler.kie.builde
INFORMACIÓN: Found kmodule: jar:file:/C:/Users/Noemi/D
jun 18, 2020 4:48:41 PM org.drools.compiler.kie.builde
INFORMACIÓN: KieModule was added: ZipKieModule[release
jun 18, 2020 4:48:42 PM com.sample.ProcessMain main
INFORMACIÓN: Process started ...
```

10. Realice el cambio de usuario a **admin** en el método obtenerUrl.

```
private static String obtenerUrl(){
    String url = "";
    try {
        KieServices ks = KieServices.Factory.get();
        KieContainer kContainer = ks.getKieClasspathContainer();
        KieSession kSession = kContainer.newKieSession("ksession-rules");

        Usuario usuario = new Usuario();
        usuario.setUsuario("admin");

        kSession.insert(usuario);
        kSession.fireAllRules();
```

11. Ejecute el aplicativo y verifique el resultado de la consola.

```
jun 18, 2020 4:58:03 PM org.jbpm.runtime.manager.impl
INFORMACIÓN: SingletonRuntimeManager is created for c
Hola api from Mule
jun 18, 2020 4:58:03 PM org.drools.compiler.kie.build
INFORMACIÓN: Found kmodule: file:/C:/Users/Noemi/Down
jun 18, 2020 4:58:04 PM org.drools.compiler.kie.build
INFORMACIÓN: KieModule was added: FileKieModule[relea
jun 18, 2020 4:58:04 PM org.drools.compiler.kie.build
INFORMACIÓN: Found kmodule: jar:file:/C:/Users/Noemi/
jun 18, 2020 4:58:04 PM org.drools.compiler.kie.build
INFORMACIÓN: KieModule was added: ZipKieModule[releas
{'descripcion': 'Laptop'}
jun 18, 2020 4:58:05 PM com.sample.ProcessMain main
INFORMACIÓN: Process started ...
```

Actividad:

1. Cree un proyecto Drools similar al primer ejemplo y modifique las reglas para verificar otros resultados.

2. Realice una integración de Drools con jBPM y ESB Mule con al menos 3 ó 4 servicios gestionados por usuarios o establezca sus propias reglas.